



SLM-Powered Agent for Visualization-Oriented Natural Language Interface: A Case Study on Grafana

Student: Bo-Yao Tong

Advisor: Prof. Chia-Yu Lin, Prof. Ted Kuo

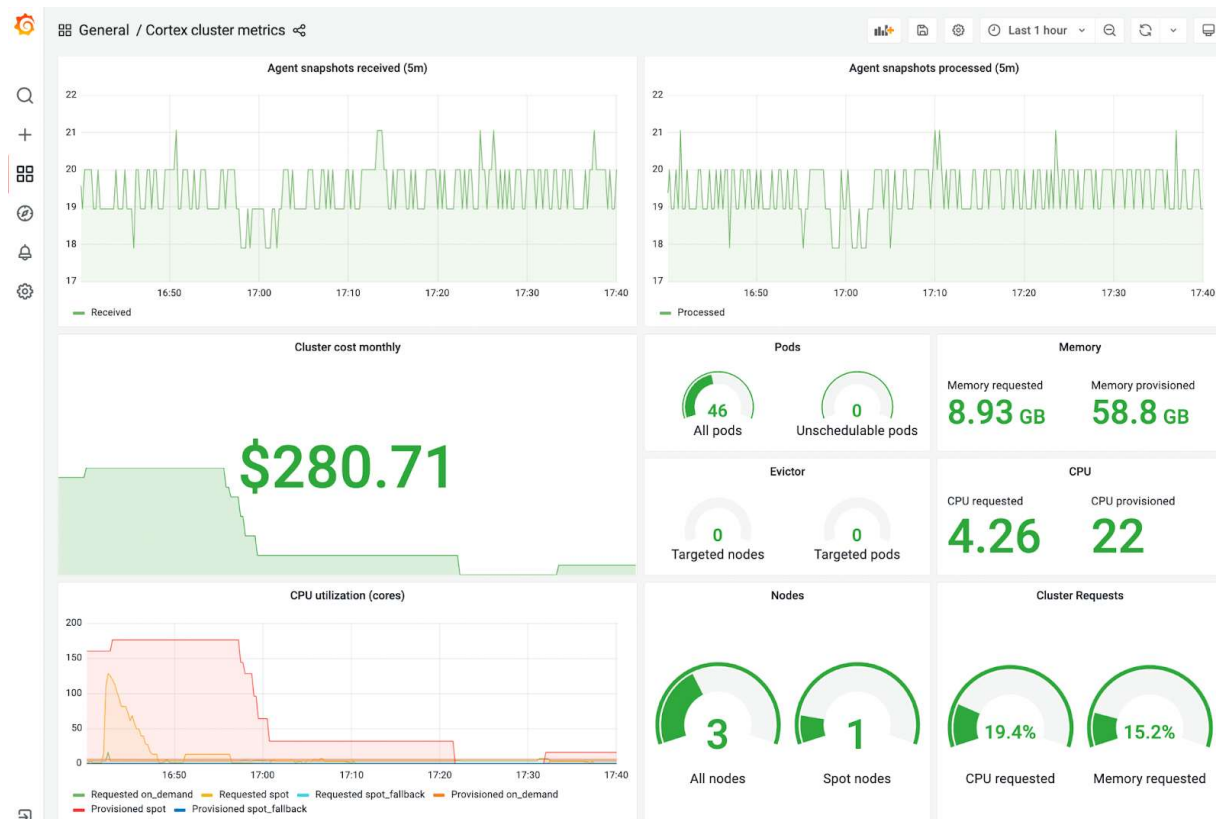
Outline



- Introduction
 - Background
 - Motivation
 - Problem Description
 - Goal
 - Contribution
- Related Works
- Method
- Experiments
- Conclusions

Background

- Grafana
 - An open-source platform for querying and visualizing metrics.
 - Supports various data sources like SQLite and Prometheus.
 - Dropdown-based configuration may not be intuitive for beginners



Background



- Steps to create a chart in Grafana:

1

Select Data Source

2

Write Query Statements

e.g. `rate(node_cpu_seconds_total{mode="user"}[5m])`

3

Set Chart Type

4

Configure Chart Parameters

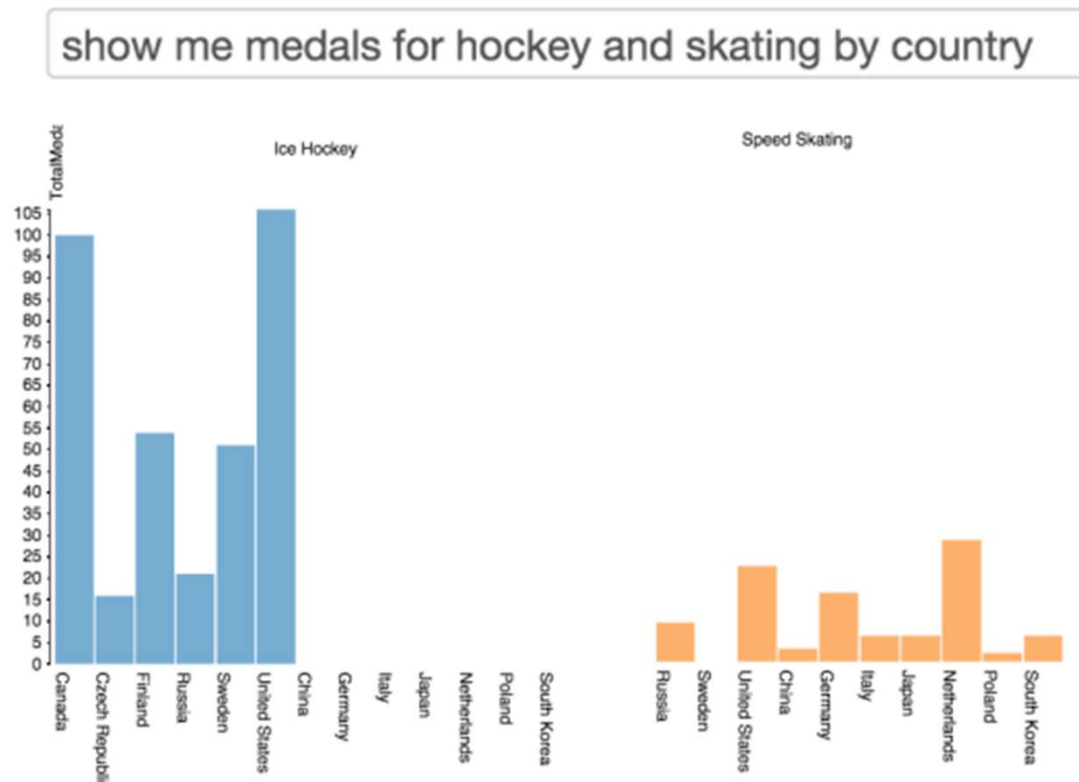
e.g. legend, thresholds, transformations, axes, panel title

5

Save and Return to Dashboard for Layout

Background

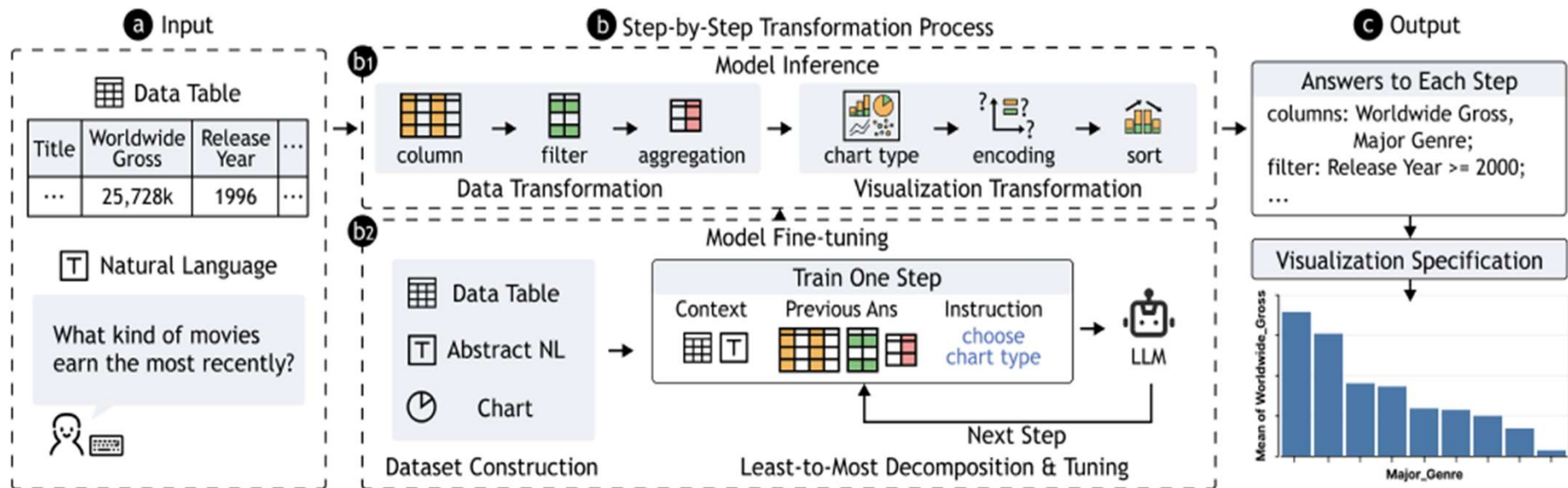
- V-NLI
 - Visualization-oriented Natural Language Interfaces
 - Generate visualizations from natural language input
 - Early V-NLI systems used rule-based methods to generate charts



1. Gao, T., Dontcheva, M., Adar, E., Liu, Z., & Karahalios, K. G. (2015, November). Datatone: Managing ambiguity in natural language interfaces for data visualization. In Proceedings of the 28th annual acm symposium on user interface software & technology (pp. 489-500).

Motivation

- Traditional interfaces are less intuitive, driving research on using large language models (LLMs) for visualization creation via natural language.
- As an V-NLI system, ChartGPT follows a structured six-step process to convert natural language into Vega-Lite JSON.



- Tian, Y., Cui, W., Deng, D., Yi, X., Yang, Y., Zhang, H., & Wu, Y. (2024). Chartgpt: Leveraging llms to generate charts from abstract natural language. IEEE Transactions on Visualization and Computer Graphics.

Problem Definition



➤ **SLM Fails to Generate Tool Call Information**

- ChartGPT has two key limitations.
 1. Fixed process – It strictly follows six predefined steps without adjusting to different user inputs.
 2. High dependency on input completeness – It fails to handle incomplete user data, often leading to incorrect or failed chart generation.
- To address these limitations, we consider using LLMs to generate tool calls, allowing more flexible visualization generation.
- Due to the high GPU resource requirements of LLMs, they are difficult to run locally.
- Instead, we use small language models (SLMs) as a lightweight alternative.
- However, SLMs are rarely trained to output tool calls in the required format.

```
"tool_calls": [{ "name": "create_chart", "arguments": {chart_type": "line"}}]
```



Problem Definition

- SLMs output tool calls will encounter these problems
 1. Incorrect format
 2. Inaccurate parameter generation
 3. Inconsistency

1st output

```
{
  "name": "create_chart",
  1 "description": "Create a Vega-Lite style chart template to visualize `tx_brat_e_downlink` for all timestamps.",
  "parameters": {
    "columns": ["Timestamp"],
    "chart_type": "line" 1 // Assuming line plot is suitable; if not, specify another type like bar or scatter.
  },
  "encodings": [
    {"field": "Timestamp", "type": "temporal"},
    {"field": "tx_brat_e_downlink", "type": "quantitative"}
  ]
}
```

1,2

2nd output

```
{
  "name": "create_chart",
  1 "description": "Create a Vega-Lite style chart template to visualize `tx_brat_e_downlink` for all timestamps.",
  "parameters": {
    "columns": ["Timestamp"],
    "chart_type": "line" 1 // Choose the appropriate type of line plot, bar graph or scatterplot.
  },
  "encodings": {
    "x": {"column": "Timestamp"},
    "y": {"column": "tx_brat_e_downlink", "type": "quantitative"}
  }
}
```

3 Inconsistency

Problem Definition



➤ Tool Call Workflow needs Flexibility

- We define the tool call input-output relationship as:

$$\{T_1, \dots, T_m\} = f(Q, \{D_1, \dots, D_n\})$$

Where

- Q : user query
- D : user data description, $n \leq 4$
- $D = \{D_1: \text{Column name}, D_2: \text{Column type}, D_3: \text{Table name}, D_4: \text{Data example}\}$
- T : Tool call, $T = \{N, P_1, \dots, P_p\}$
 - N: Tool name
 - P: Tool parameters, where p is fixed for each tool
- Take ChartGPT for example, where n is fixed at 4, and m is fixed at 6.
- As user data may be incomplete ($n < 4$), the system must address missing fields based on the available information.
- Not all tools ($m < 6$) may be required for straightforward user requests, such as aggregation and filtering tools.

Problem Definition



- Therefore, we need an agent to handle data incompleteness and dynamic tool selection.
- In addition, this agent needs to parse and execute tool calls to achieve automated visualization generation.

➤ **Lack of Tool Call Integration in Visualization Tasks**

- To support dynamic tool selection, we need many tools with different functions to be called.
- Most existing tools are designed for general-purpose tasks, and only a few are specifically designed for automated visualization.
- Therefore, we design tools that enable SLM and the agent to execute visualization tasks more flexibly.

Goal



- Using Grafana as an example, we propose an SLM-driven agent workflow as the core design of our V-NLI system.
 - Develop a toolset that enable SLMs to create and update visualizations in Grafana.
 - Fine-tune the SLMs to improve the accuracy of tool parameters generation.
 - Design an agent to handle data incompleteness and dynamic tool selection

Contribution



- Our fine-tuned Phi-3.5 achieves better performance than ChartGPT, with a $\sim 1\%$ improvement in ROUGE-L and comparable BLEU scores on public datasets.
- Furthermore, compared to xLAM, LLaMA3.2, and Qwen2.5, our model achieves an average improvement of around 78% in ROUGE-L and 48% in BLEU.
- Robust on private data, generating correct visualizations even with incomplete input.
- Introduced a platform-agnostic Tool Call workflow, combining SLM for generation and Agent for execution, and successfully deployed it on Grafana.



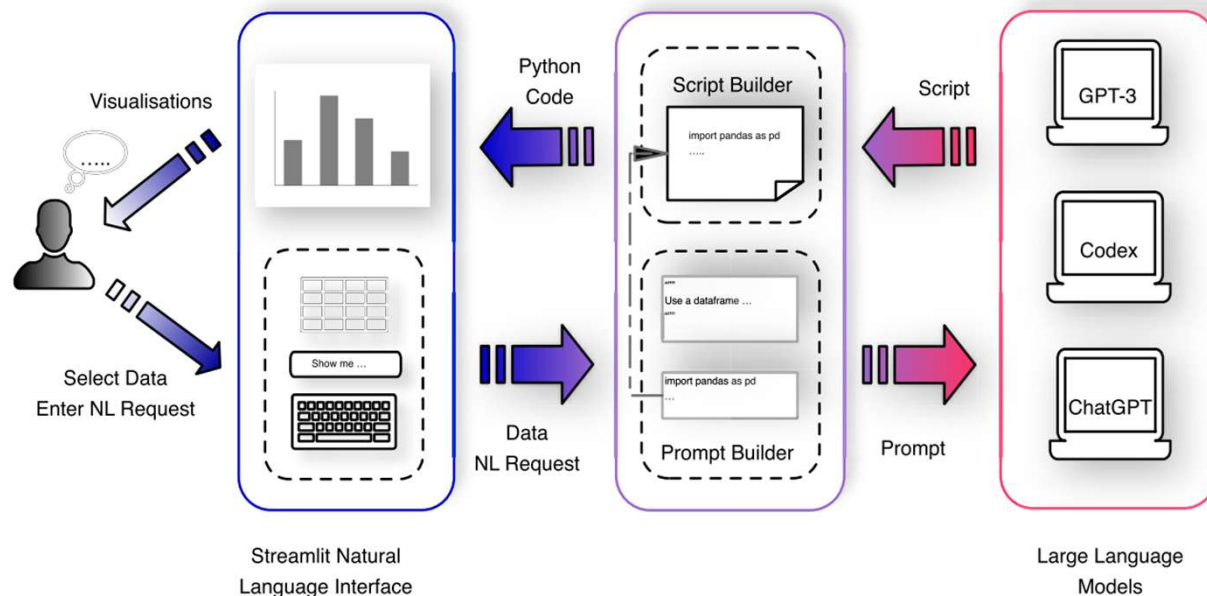
Related Works



Integrating LLMs into V-NLI

Related Works

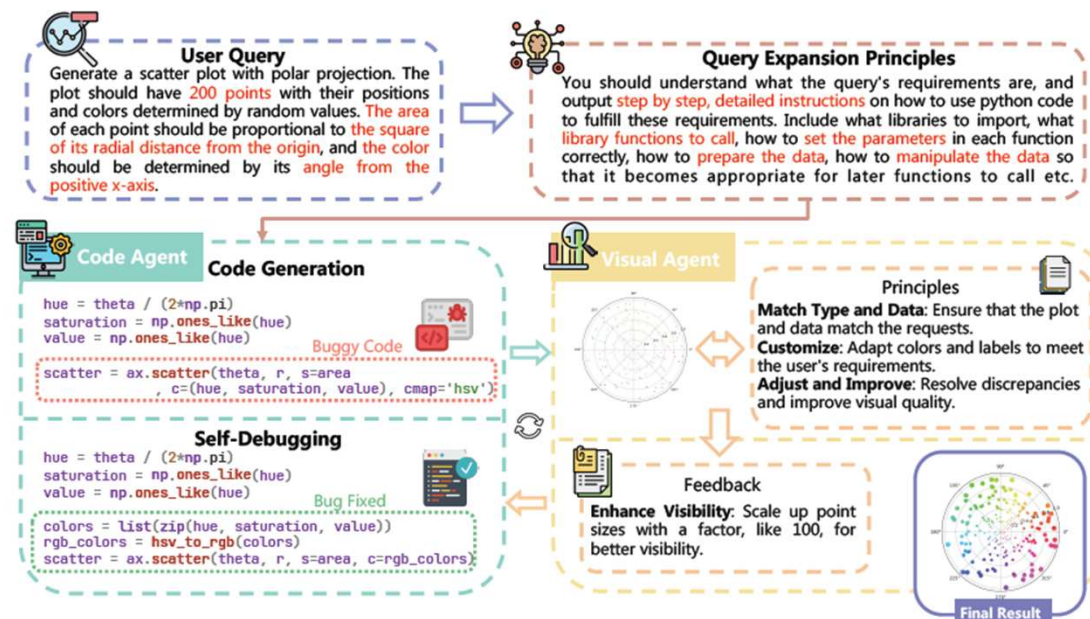
- **Title:** Chat2VIS: Generating Data Visualizations via Natural Language Using ChatGPT, Codex and GPT-3 Large Language Models
- **Challenge:** Addressing the ambiguity of natural language.
- **Approach:** Utilizing prompt engineering (Description Prompt + Code Prompt) to generate python scripts for visualization code.
- **Limitation:** Difficulty in consistently applying visual settings due to diverse chart types and parameter compatibility issues.



1. Maddigan, P., & Susnjak, T. (2023). Chat2vis: Generating data visualizations via natural language using chatgpt, codex and gpt-3 large language models. *Ieee Access*, 11, 45181-45193.

Related Works

- **Title:** MatPlotAgent: Method and Evaluation for LLM-Based Agentic Scientific Data Visualization
- **Challenge:** Existing LLMs struggle to generate accurate visualizations, especially when user queries are complex or ambiguous.
- **Approach:** MatPlotAgent adopts a three-stage agent framework—query expansion, code generation with self-debugging, and visual feedback
- **Limitation:** The system heavily relies on GPT-4 and GPT-4V, without considering deployment cost or feasibility for real-world applications.



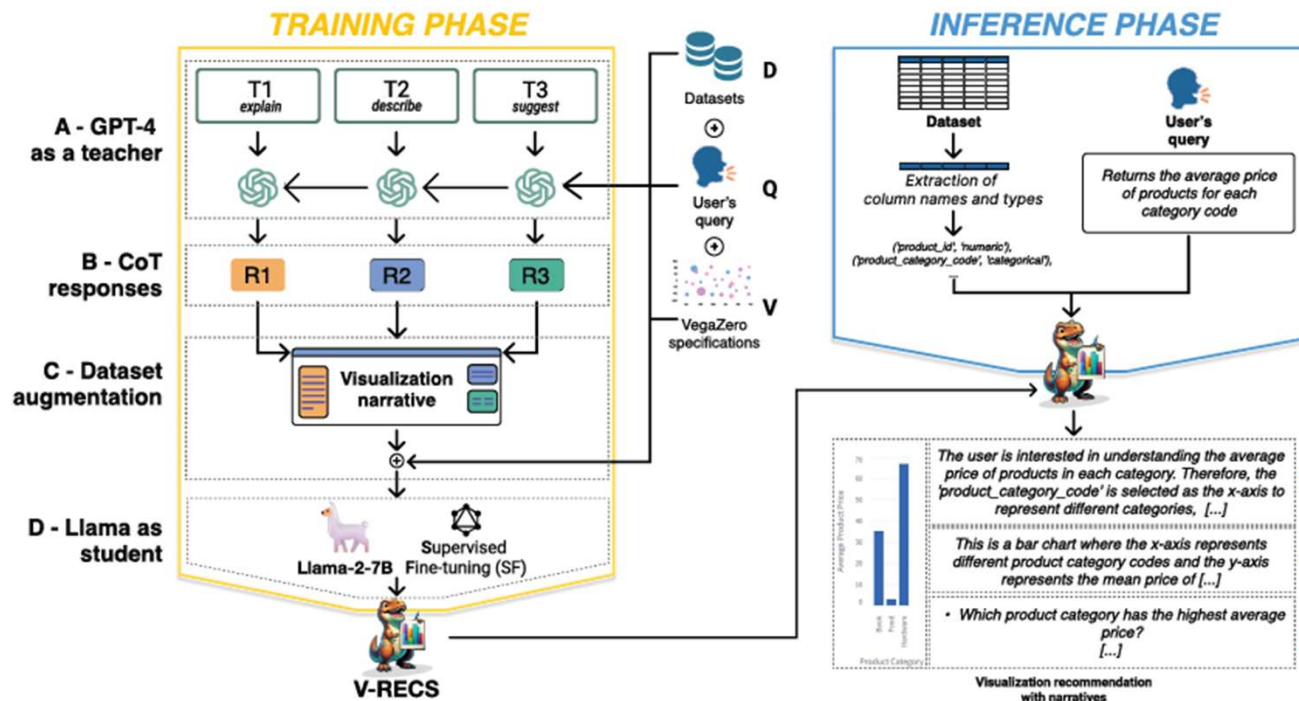
1. Yang, Z., Zhou, Z., Wang, S., Cong, X., Han, X., Yan, Y., ... & Sun, M. (2024). Matplotlibagent: Method and evaluation for llm-based agentic scientific data visualization. arXiv preprint arXiv:2402.11453.



Integrating SLMs into V-NLI

Related Works

- **Title:** V-RECS, a Low-Cost LLM4VIS Recommender with Explanations, Captioning, and Suggestions
- **Challenge:** LLMs are costly and often impractical for deployment in real-world systems.
- **Approach:** V-RECS uses GPT-4 to generate Explanations, Captions, and Suggestions (E/C/S), then fine-tunes a smaller LLaMA-2-7B model for chart and narrative generation.
- **Limitation:** Evaluation only compares with GPT-4, lacking broader benchmarking, and provides no quantitative validation for the accuracy or usefulness of generated narratives.

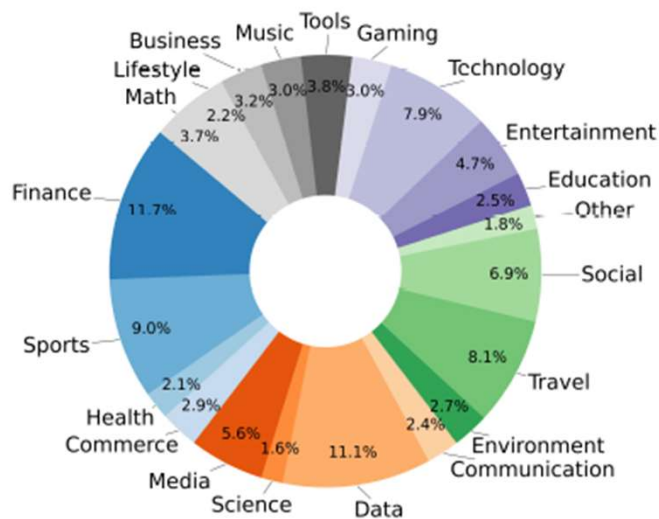




Tool Call

Related Works

- **Title:** APIGen: Automated Pipeline for Generating Verifiable and Diverse Function-Calling Datasets
- **Challenge:** Improving dataset quality for training function-calling language model to handle real-world API interactions.
- **Approach:** Proposes APIGen to generate high-quality datasets with a multi-stage verification process (format checking, execution validation, and semantic alignment) to ensure reliability and diversity in function-calling tasks.
- **Limitation:** Currently handles only single-turn function calling, and lacks sufficient data for visualization tasks, leading to less accurate results when applied to this domain.



APIs

```
{
  "name": "api_name",
  "description": "API description",
  "parameters": {
    "param_name": {
      "type": "data type",
      "description": "",
      "default": "",
      "required": true or false
    },
    ... (more parameters)
  }
}
```

Function Call

```
{
  {
    "name": "api_name",
    "arguments": {
      "arg_name1": "value1",
      "arg_name2": "value2",
      ... (more arguments)
    }
  }
  ... (more API calls)
}
```

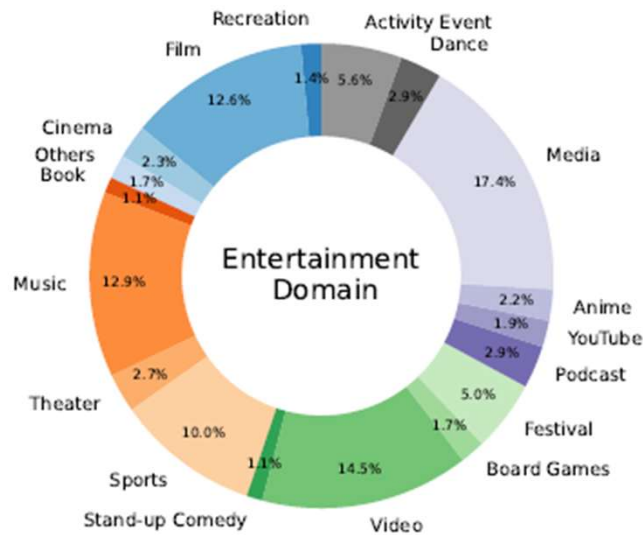
Generator Output

```
{
  "query": "The generated query.",
  "answers": [
    {
      "name": "api_name",
      "arguments": {
        "arg_name": "value",
        ... (more arguments)
      }
    }
    ... (more API calls)
  ]
}
```

1. Liu, Z., Hoang, T., Zhang, J., Zhu, M., Lan, T., Tan, J., ... & Xiong, C. (2024). Apigen: Automated pipeline for generating verifiable and diverse function-calling datasets. Advances in Neural Information Processing Systems, 37, 54463-54482.

Related Works

- **Title:** ToolACE: Winning the Points of LLM Function Calling
- **Challenge:** Existing datasets are too simple and narrow for real-world function calling.
- **Approach:** ToolACE uses self-evolving API synthesis and a Multi-Agent Interactive (MAI) pipeline to generate diverse, complex dialogs.
- **Limitation:** The dataset lacks domain-specific precision, making it less reliable for high-accuracy visualization tasks.



Algorithm 1 MAI Dialog Generation

```

1: Initialization: Sampled API list  $A$ , Dialog  $D_0 = []$ , Target Turn Length  $N_t$ 
2: Definition: User Agent  $\theta_u$  and its output  $C_u$ , Assistant Agent  $\theta_a$  and its output  $C_a$ , Tool Agent  $\theta_t$  and its output  $C_t$ 
3: for  $t = 1, 2, \dots, N_t$  do
4:   Sample user template  $p$  and user style  $s$ 
5:    $C_u = \theta_u(D_{t-1}, A, p, s)$ 
6:    $C_a^1, C_a^2, \dots, C_a^N = \theta_a(C_u, D_{t-1}, A)$   $\triangleright$  Generate response N times with formalized thinking.
7:   if  $C_a^1 \neq C_a^2 \neq \dots \neq C_a^N$  then  $\triangleright$  Only check consistency of tool callings.
8:     Continue or Add Loss Mask  $\triangleright$  Discard the turn or add a loss mask to the turn.
9:   else
10:     $C_a = \text{MajorVote}(C_a^1, C_a^2, \dots, C_a^N)$ 
11:   end if
12:    $D_t = D_{t-1} + [C_u, C_a]$ 
13:   while Tool calling in  $C_a$  do  $\triangleright$  Dependent functions need several sequential tool callings.
14:      $C_t = \theta_t(C_a, A)$ 
15:      $C_a = \theta_a(C_t, D_t, A)$ 
16:      $D_t = D_t + [C_t, C_a]$ 
17:   end while
18: end for  $\triangleright$  The generated dialog  $D_{N_t}$  can be further transformed to diverse formats.

```

Comparison

- SLM: Refers to models with fewer than 10 billion parameters.
- Visualization Quality: Indicates whether the system is specifically designed or optimized for visualization tasks.
- Robustness: Handles incomplete but semantically similar inputs while producing consistent results.

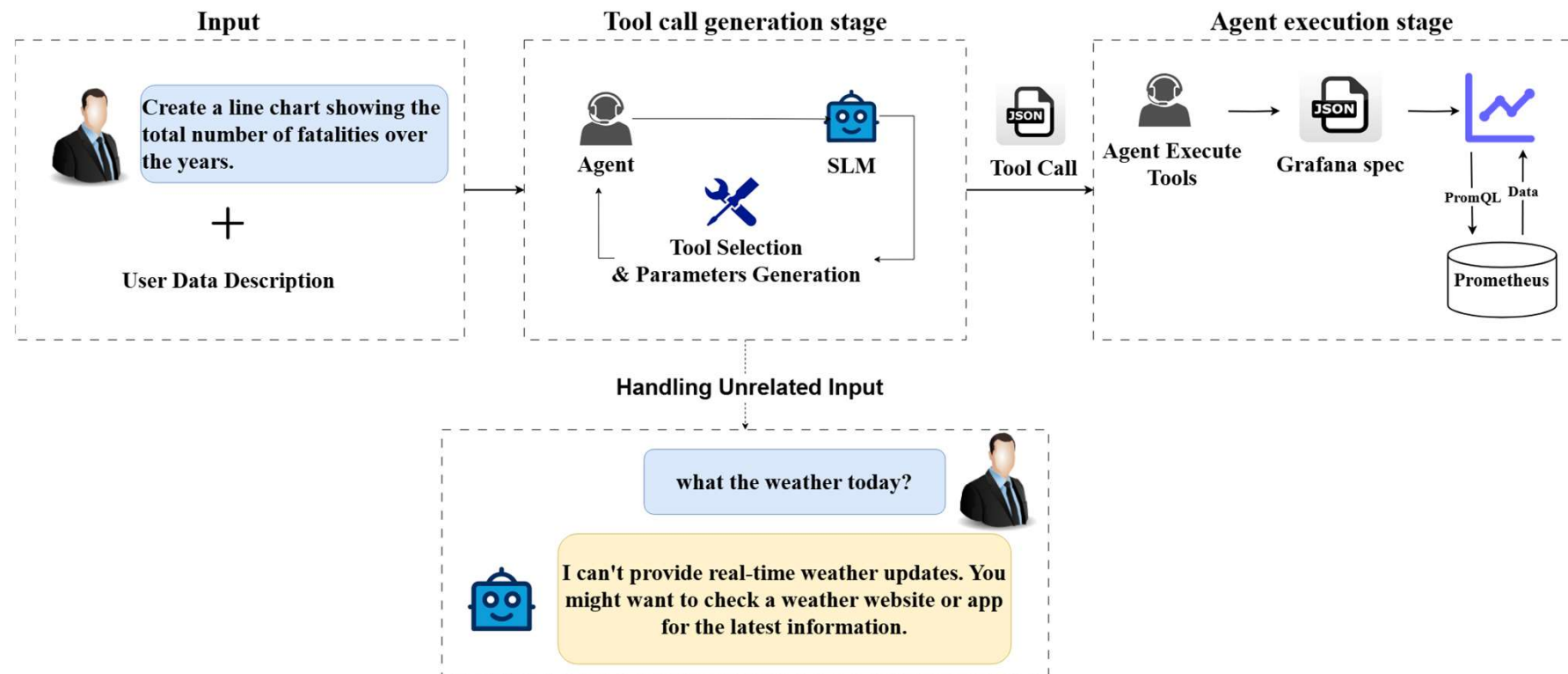
System	Base Model	SLM	Visualization Quality	Robustness
Chat2vis	GPT-3	✗	✓	✗
MatplotAgent	GPT-4,GPT-4V	✗	✓	✓
ChartGPT	Flan-T5-XL	✓	✓	✗
xLAM	DeepSeek-Coder	✓	✗	✓
ToolACE	Llama-3.1	✓	✗	✓
Ours	Phi-3.5	✓	✓	✓



Method

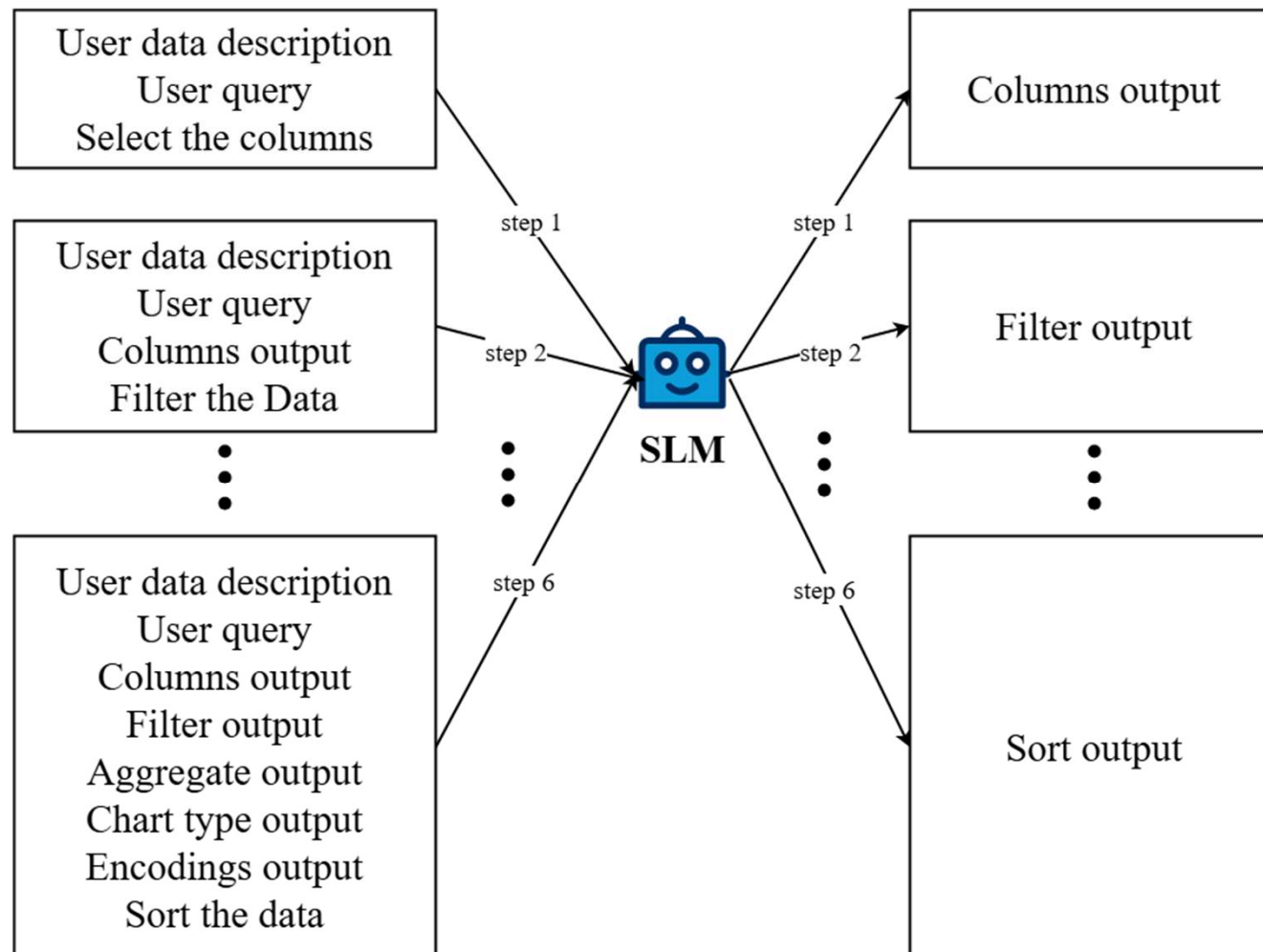
System Overview

- SLM-Powered Agent for Visualization-Oriented Natural Language Interface



Training Workflow

- Step-by-Step Training Workflow for Accurate Parameter Selection



SLM Fine-tuning



- To improve parameter selection accuracy, we fine-tune the SLM to better understand which parameters are required.
- We use ChartGPT dataset to fine-tune Phi3.5
 - Training Data: 9.23 rows
 - Test Data: 2.27 rows
 - Model Input Data
 - Table structure (name, column names, column types)
 - Sample data rows
 - Natural language query
 - Step-by-step prompts
 - Previous step answer
 - Model Output: Step answer

Training Data Example



prompt	completion
Faculty <head>FacID,Lname,Fname,Rank,Sex,Phone,Room,Building <type>quantitative,nominal,nominal,nominal,nominal,quantitative,nominal,nominal <data> 1082,Giuliano,Mark,Instructor,M,2424,224,NEB <line> 1121,Goodrich,Michael,Professor,M,3593,219,NEB <line> <utterance> What does the composition of faculty members look like based on their roles? <ans> <sep> Step 1. Select the columns:	Rank
Step 2 ~5	
Same as Step 1 <sep> Step 1. Select the columns: Rank <sep> Step 2. Filter the data: none <sep> Step 3. Add aggregate functions: count Rank <sep> Step 4. Choose chart type: arc <sep> Step 5. Select encodings: x=Rank,y=count Rank,color=none <sep> Step 6. Sort the data:	none

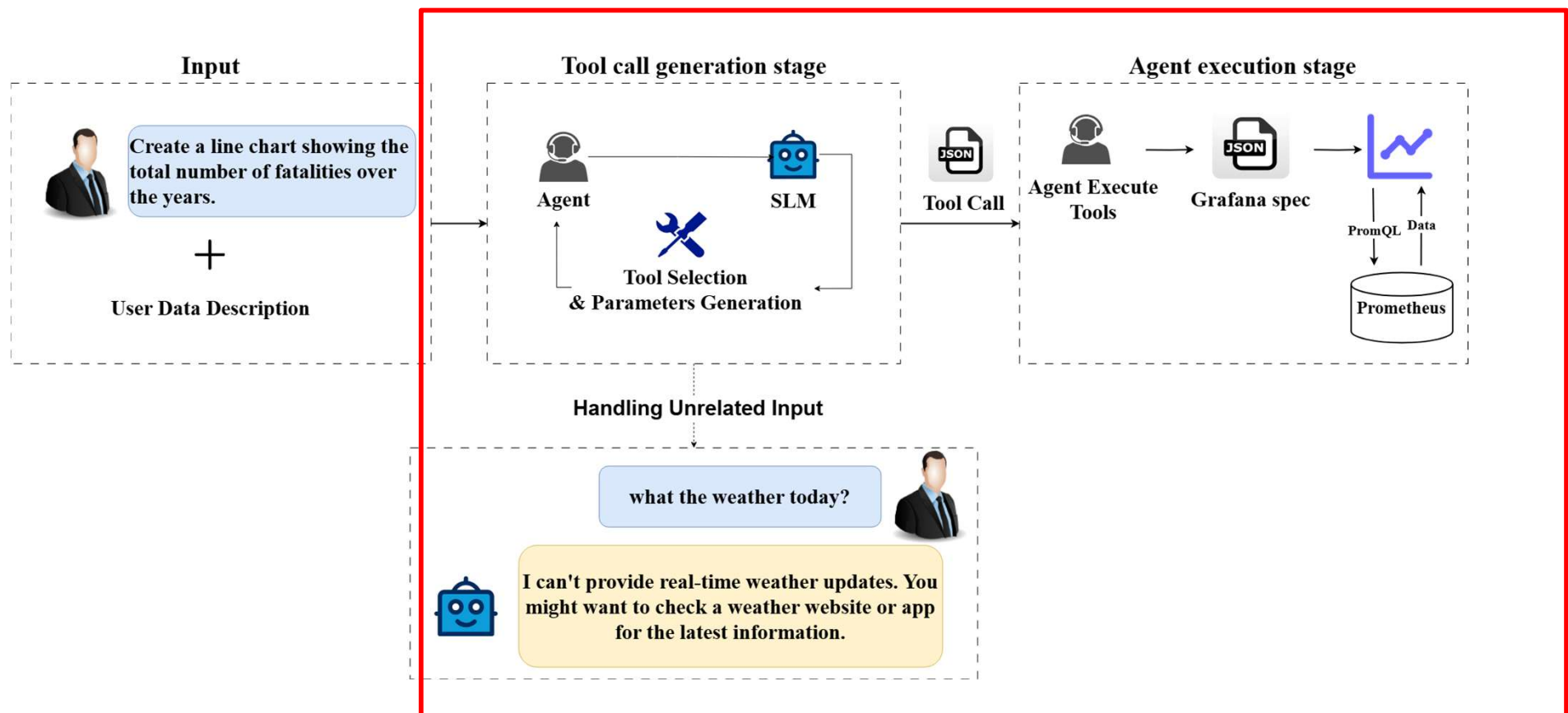
Fine-tune SLM



- Step 1-6 prompts:
 - Select columns
 - Add filter
 - Add aggregations
 - Select chart type
 - Choose encoding
 - Add sort
- Training hyperparameters
 - Batch size: 8
 - Gradient accumulation steps: 2
 - Warmup steps: 50
 - Epochs: 3
 - Learning rate: 1e-4
 - Optimizer: AdamW (8-bit)

Inference Workflow

- We use an agent to guide the SLM in generating tool calls step by step.



Tool Design



- To avoid unnecessary execution, we distribute parameters like sorting and aggregation across different tools.
- Three tools and Parameter Allocation
 - 1. Basic Chart (`create_chart`)**
 - Purpose: Defines the fundamental chart type and display settings.
 - Parameters: column, encoding, chart
 - Integration: Sets the X-axis, Y-axis, and visualization type.
 - 2. Data Processing (`apply_transformation`)**
 - Purpose: Handles data transformations such as sorting and aggregation.
 - Parameters: aggregate, sort
 - Integration: Similar to Transform, enabling Group by, Sort by, and aggregate calculations like mean or sum.

Tool Design



3. Filtering (`set_variable`)

- Purpose: Adjusts data range or modifies Grafana dashboard variables.
- Parameters: filter
- Integration: Tools similarly to dashboard variables, allowing dynamic data filtering without modifying the query.

4. SQL Generation (`generate_sql`)

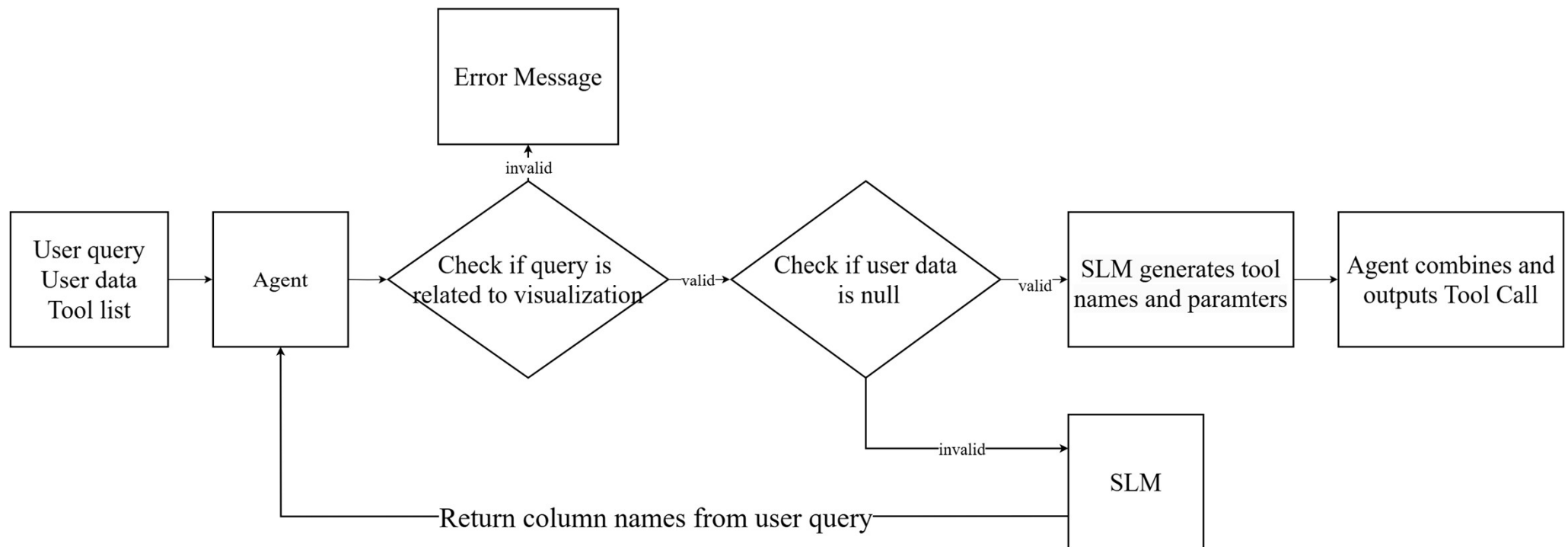
- Purpose: Constructs PromQL to retrieve relevant data from the database.
- Parameters: columns, filter, aggregate.
- Integration: Enables dynamic data extraction based on user queries.

5. API Update (`update_dashboard`)

- Purpose: Updates an Grafana dashboard by adding visualizations.
- Parameters: x
- Integration: Since we embed the Rasa framework into a specific Grafana dashboard, the final chart is automatically uploaded to and displayed on that dashboard.

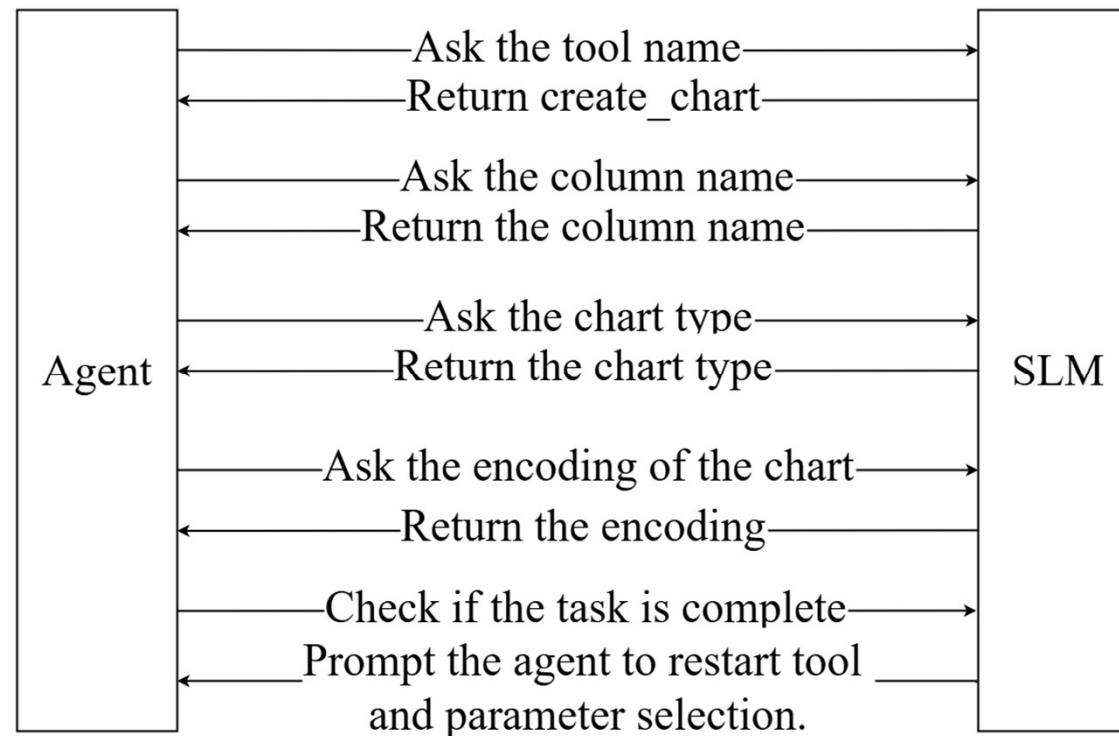
Validation Stage

- With an agent, we can control the tool call output logically, ensuring correct formatting and structure.
- Column names are essential for retrieving and visualizing data.



Generation Stage

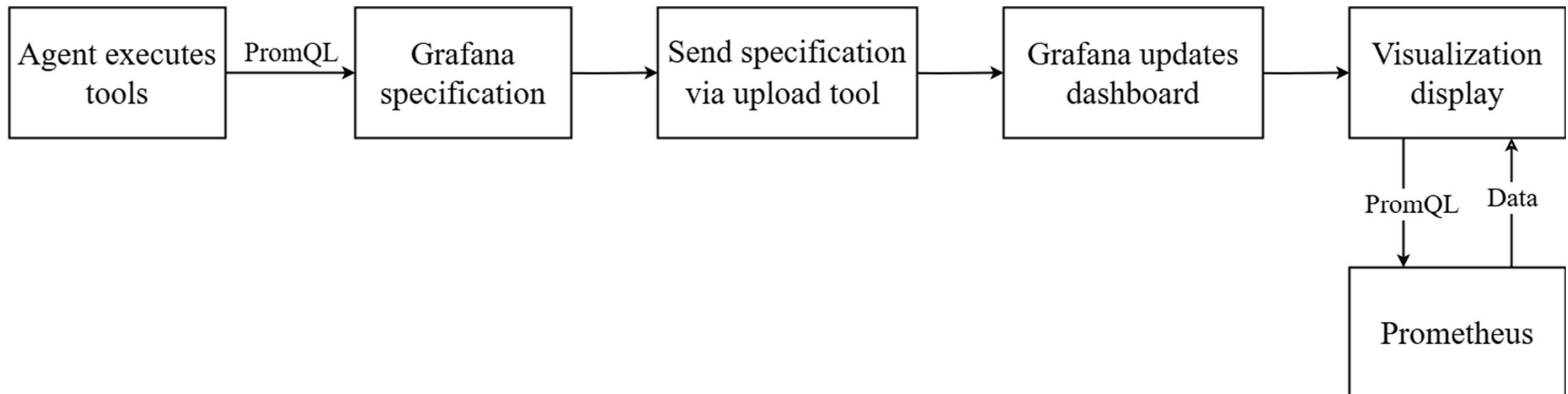
- Despite our efforts in fine-tuning SLM on tool calls, its parameter generation is still inaccurate.
- To address this, we employ an agent to guide the SLM step-by-step in generating tool calls.
- Example of calling `create_chart`:





Execution Stage

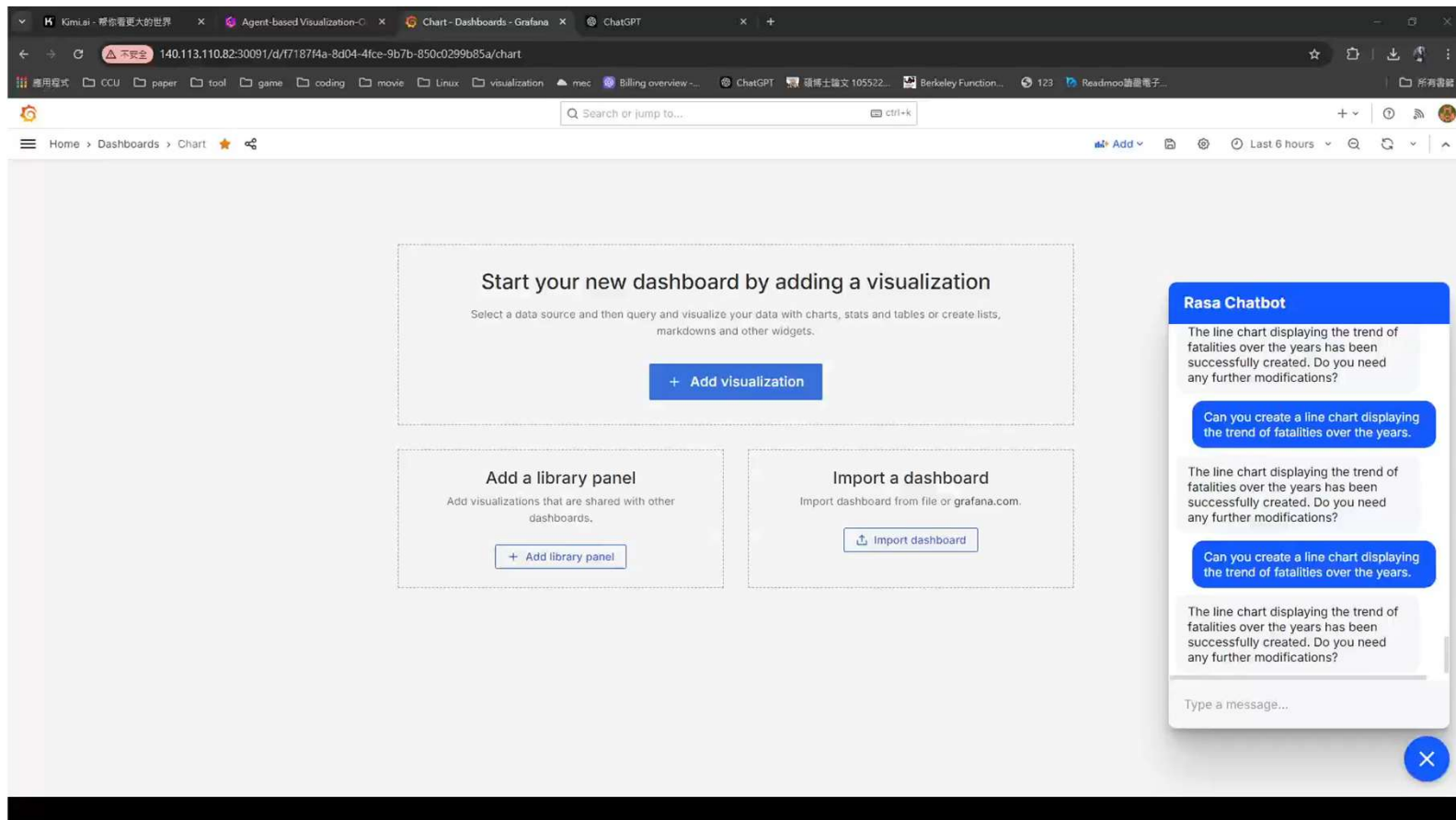
- The agent ensures the tool call is correctly executed, sending the visualization spec to Grafana for rendering.





Demo

- Query: Can you create a line chart displaying the trend of fatalities over the years.





Experiment

Experiment



- Experiment 1 - Comparision with ChartGPT
- Experiment 2 – Robustness testing

Evaluation



- Dataset: ChartGPT testset
- ROUGE-L (Recall-Oriented Understudy for Gisting Evaluation - Longest Common Subsequence)
 - Usage Context:
 - Used in **machine translation**.
 - Assesses how well the generated text captures the essence of the source or reference texts.
 - Represents:
 - Highlights the **completeness** of the content captured in the generated output.

Evaluation

- ROUGE-L
 - Recall-Oriented Understudy for Gisting Evaluation

$$\text{ROUGE-L} = \frac{(1 + \beta^2) \cdot R_{\text{lcs}} \cdot P_{\text{lcs}}}{R_{\text{lcs}} + \beta^2 \cdot P_{\text{lcs}}}$$

$$P_{\text{lcs}} = \frac{LCS(X, Y)}{|X|} \quad R_{\text{lcs}} = \frac{LCS(X, Y)}{|Y|}$$

Evaluation



- LCS(X,Y)
 - The Longest Common Subsequence (LCS) measures the longest ordered sequence of words shared between two sentences.

R. police killed the gunman

S1. police kill the gunman

S2. the gunman kill police

ROUGE-L(S1) = $3/4 = 0.75$ (police the gunman)

ROUGE-L(S2) = $2/4 = 0.5$ (the gunman)

Evaluation



- BLEU (Bilingual Evaluation Understudy)
 - Usage Context:
 - Used in **machine translation**.
 - Evaluates the quality of text translated by machines against human-generated translations.
 - Represents:
 - Focuses on the **accuracy and fluency** of translated words and phrases.

Evaluation

- BLEU
 - N-gram + clipped count

1-gram	Reference	Candidate
1	The	The
2	Cat	Cat
3	Is	The
4	On	Cat
5	The	On
6	Mat	The
7		Mat

Reference: the cat is on the mat.

Candidate: the cat the cat on the mat.

1-gram precision: 5/7



Evaluation

- BLEU
 - Bilingual Evaluation Understudy

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^1 w_n \cdot \log p_n \right)$$

$$\text{BP} = \begin{cases} 1 & \text{if } c > r \\ e^{(1-\frac{r}{c})} & \text{if } c \leq r \end{cases}$$

Evaluation Process



- Generate 6 parameters
 - The six parameters from ChartGPT are: column, aggregate, filter, chart, decoding, color.
 - These parameters are concatenated into a single text sequence for evaluation.
 - Example: `Rank none count arc x=Rank,y=count_Rank,color=none none`
- Evaluation Metrics
 - Rouge-L
 - 1-gram BLEU



Result

- Final Performance After Adding the Agent
 - After incorporating the agent, Phi 3.5 achieved the highest Rouge-L score, surpassing ChartGPT by 0.1.
 - BLEU scores remained similar for the top models, with Phi 3.5 and ChartGPT both achieving 0.91.

Method	Model	Model Size	ROUGE-L	BLEU
Baseline	xLAM	7B	0.41	0.54
	LLaMA3.2	3B	0.56	0.65
	Qwen2.5	7B	0.54	0.67
Fine-tuned	ChartGPT	2.85B	0.87	0.91
	Ours	3.8B	0.88	0.91

Experiment



- Experiment 1 - Comparision of ChartGPT
- Experiment 2 - Add custom option on ChartGPT Dataset

6G Dataset



- Dataset: colosseum-oran-commag
- We utilize data from the rome_static_close scenario, focusing on four base stations (BSs) with network slicing.
 - Static: UEs remain stationary throughout the experiment.
 - Close: UEs are uniformly distributed within 20 meters of each BS.
- Network Slicing
 - Each BS has 3 slices, managing different traffic types:
 - Slice 0: eMBB (high throughput)
 - Slice 1: MTC (IoT traffic)
 - Slice 2: URLLC (low latency)
- Our project focuses on visualizing and analyzing prediction errors in downlink transmission rates (tx_brate_downlink), where future values are predicted based on past observations.

Database



- Our database stores CSV files for each UE, organized by four base stations.
 - bs1
 - 1010123456002_metrics.csv
 - 1010123456003_metrics.csv
 - ...
- Example CSV file

Prometheus Alerts Graph Status ▾ Help	
tx_brat_downlink(bs="1", instance="140.113.110.82:58111", job="oran_bs1", slice_id="0", time="0.0", type="Ground Truth", ue="2")	0
tx_brat_downlink(bs="1", instance="140.113.110.82:58111", job="oran_bs1", slice_id="0", time="0.0", type="Ground Truth", ue="5")	0
tx_brat_downlink(bs="1", instance="140.113.110.82:58111", job="oran_bs1", slice_id="0", time="0.0", type="Ground Truth", ue="8")	0
tx_brat_downlink(bs="1", instance="140.113.110.82:58111", job="oran_bs1", slice_id="0", time="0.0", type="mean", ue="2")	0
tx_brat_downlink(bs="1", instance="140.113.110.82:58111", job="oran_bs1", slice_id="0", time="0.0", type="mean", ue="5")	0
tx_brat_downlink(bs="1", instance="140.113.110.82:58111", job="oran_bs1", slice_id="0", time="0.0", type="mean", ue="8")	0
tx_brat_downlink(bs="1", instance="140.113.110.82:58111", job="oran_bs1", slice_id="0", time="0.25", type="Ground Truth", ue="2")	0
tx_brat_downlink(bs="1", instance="140.113.110.82:58111", job="oran_bs1", slice_id="0", time="0.25", type="Ground Truth", ue="5")	0
tx_brat_downlink(bs="1", instance="140.113.110.82:58111", job="oran_bs1", slice_id="0", time="0.25", type="Ground Truth", ue="8")	0.550432
tx_brat_downlink(bs="1", instance="140.113.110.82:58111", job="oran_bs1", slice_id="0", time="0.25", type="mean", ue="2")	0
tx_brat_downlink(bs="1", instance="140.113.110.82:58111", job="oran_bs1", slice_id="0", time="0.25", type="mean", ue="5")	0
tx_brat_downlink(bs="1", instance="140.113.110.82:58111", job="oran_bs1", slice_id="0", time="0.25", type="mean", ue="8")	0.275216
tx_brat_downlink(bs="1", instance="140.113.110.82:58111", job="oran_bs1", slice_id="0", time="0.5", type="Ground Truth", ue="2")	0
tx_brat_downlink(bs="1", instance="140.113.110.82:58111", job="oran_bs1", slice_id="0", time="0.5", type="Ground Truth", ue="5")	0
tx_brat_downlink(bs="1", instance="140.113.110.82:58111", job="oran_bs1", slice_id="0", time="0.5", type="Ground Truth", ue="8")	0.987264

Scenario

- Q_i represents whether the user query contains the chart type.
- D_i indicates whether the user data provides complete information.
 - D_1 : Column name, D_2 : Column type, D_3 : Table name, D_4 : Data example

	Q_i	D_i
Level 1	✓	D_{1-4}
Level 2	x	D_{1-4}
Level 3	x	D_1
Level 4	x	x

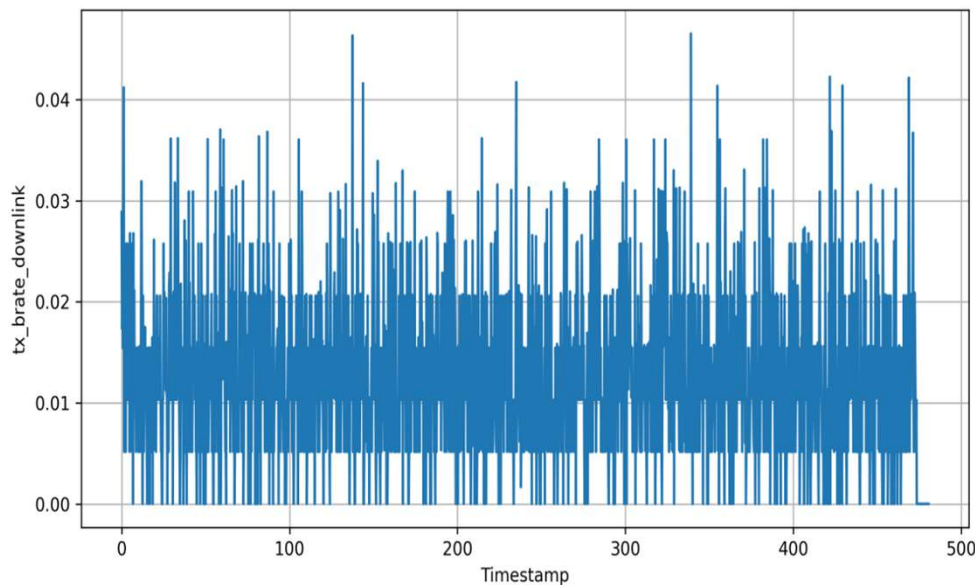
Test example



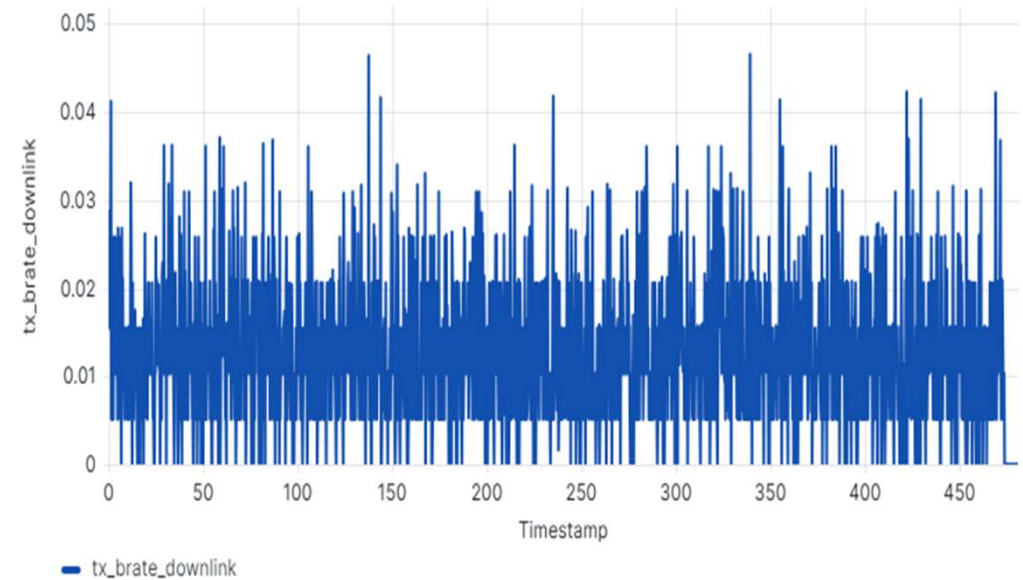
	Q _i	D _i
Level 1	Create a line chart for tx_brate_downlink over all timestamps.	5g_slice_performance <head>timestamp, slice_id ,tx_brate_downlink <type>quantitative, nominal, quantitative <data>0, 1, 0.0288 <line>0.25, 1, 0.0173<line>
Level 2	Show tx_brate_downlink for all timestamps.	5g_slice_performance <head>timestamp, slice_id ,tx_brate_downlink <type>quantitative, nominal, quantitative <data>0, 1, 0.0288 <line>0.25, 1, 0.0173<line>
Level 3	Show tx_brate_downlink for all timestamps.	<head>timestamp, slice_id ,tx_brate_downlink
Level 4	Show tx_brate_downlink for all timestamps.	×

Level 1

- Q_i : Create a line chart for tx_brate_downlink over all timestamps.
- $D = \{D_1 : \text{Column name}, D_2 : \text{Column type}, D_3 : \text{Table name}, D_4 : \text{Data example}\}$



ChartGPT

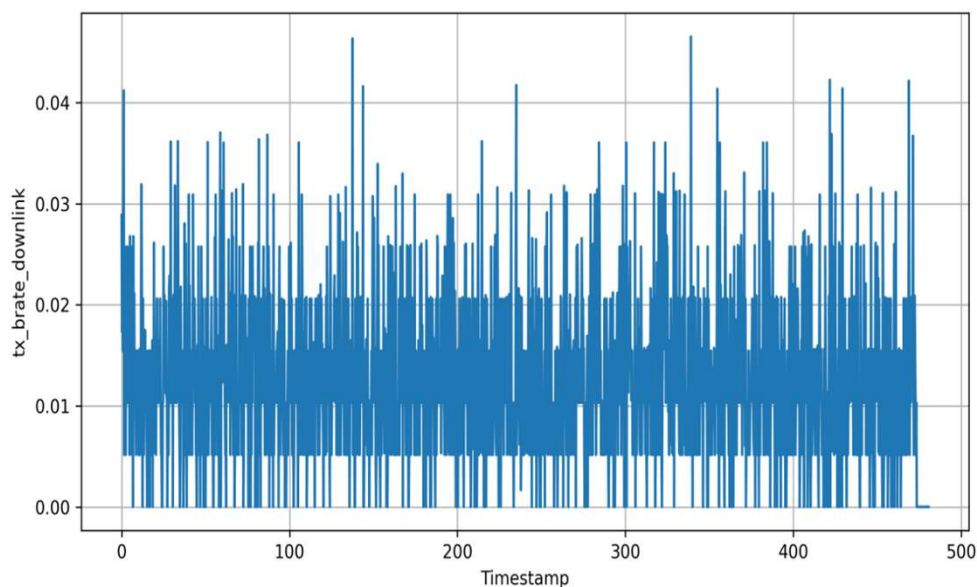


Ours

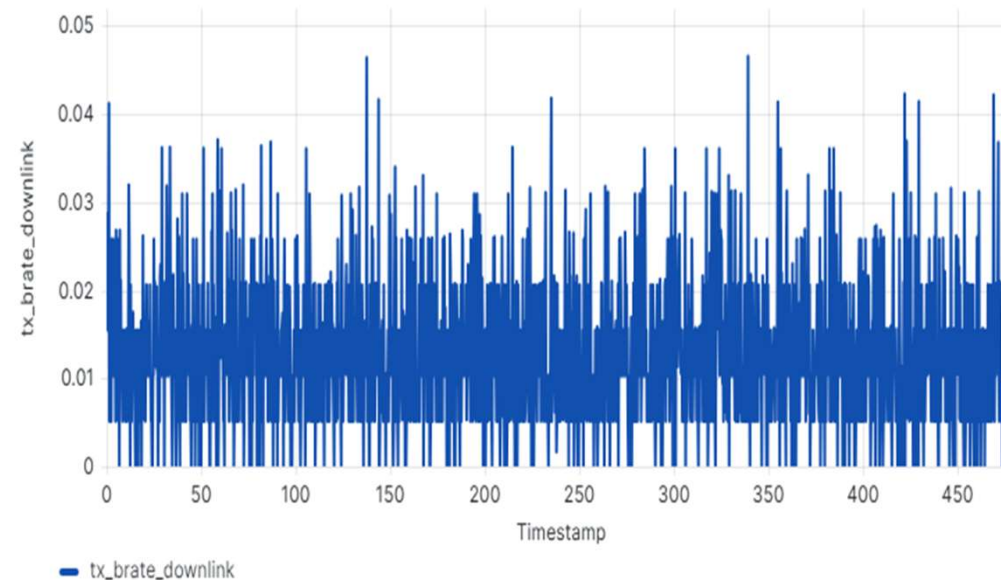
Level 2



- Q_i : Show tx_brate_downlink for all timestamps.
- $D = \{D_1 : \text{Column name}, D_2 : \text{Column type}, D_3 : \text{Table name}, D_4 : \text{Data example}\}$



ChartGPT

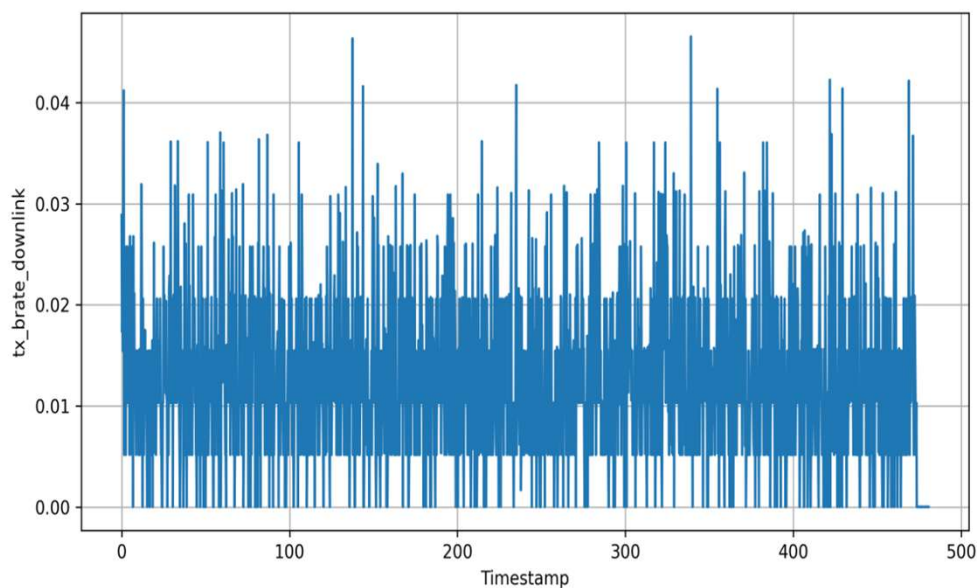


Ours

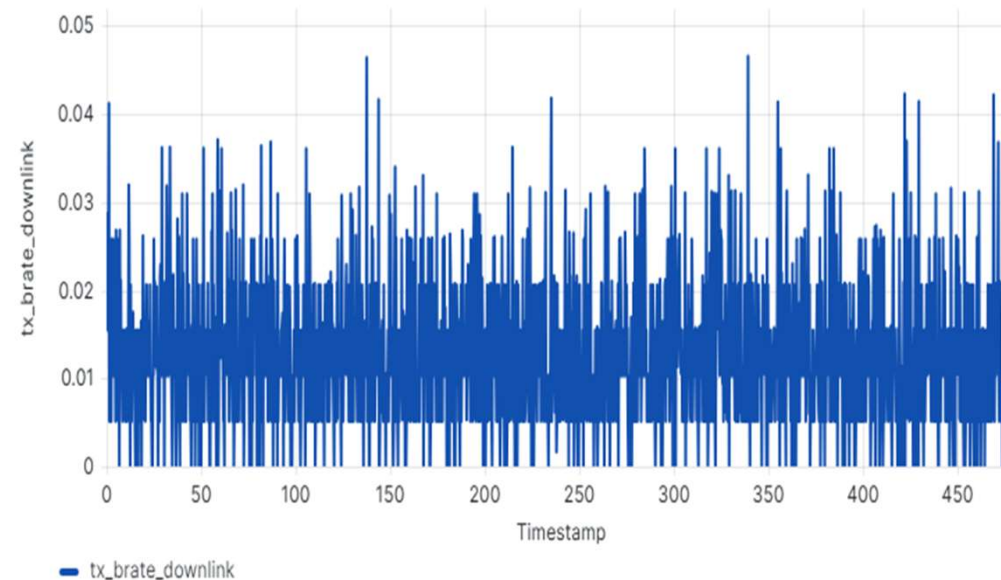
Level 3



- Q_i : Show tx_brate_downlink for all timestamps.
- $D = \{D_1 : \text{Column name}\}$



ChartGPT



Ours

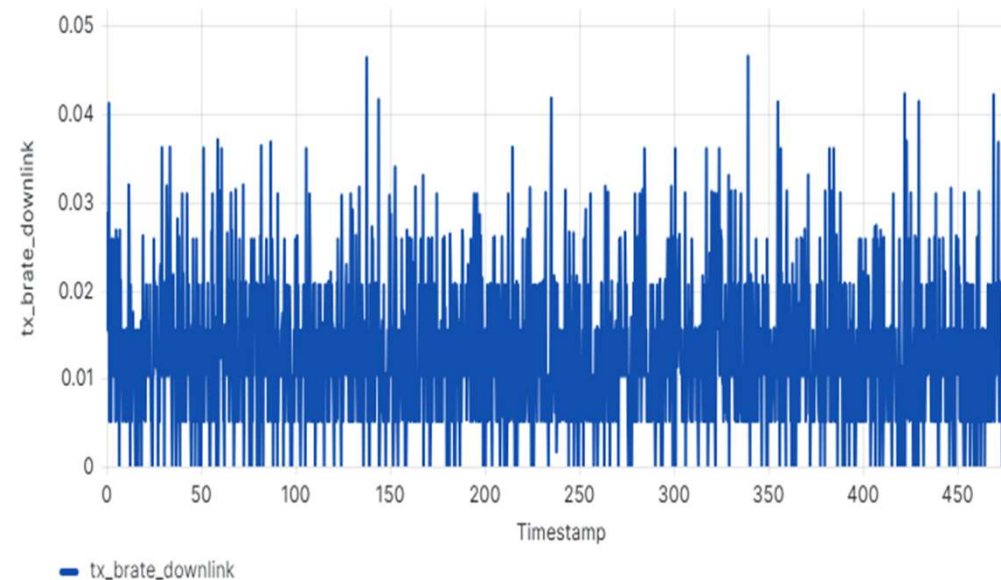
Level 4



- Q_i : Show tx_brate_downlink for all timestamps.
- $D = \{\}$

```
{  
  "mark": "line",  
  "encoding": {  
    "x": {  
      "field": "timestamp",  
      "type": "quantitative"  
    },  
    "y": {  
      "field": "tx_brate_downlink",  
      "type": "quantitative"  
    }  
  }  
}
```

ChartGPT



Ours

Conclusion



- We propose an SLM-driven agent system that generates and executes tool calls for visualization
- Experiment 1 shows that our Phi-3.5 model achieves higher ROUGE-L (0.88) and matches ChartGPT on BLEU (0.91), proving its accuracy in parameter generation.
- Experiment 1 shows that our model significantly outperforms general-purpose SLMs, with an average of +78% ROUGE-L and +48% BLEU.
- Experiment 2 confirms our system's robustness under incomplete input. It outperforms ChartGPT in the most challenging (Level 4) scenario.



Thanks